An AI agent that works on your code, on your machine.

DiveeOI is a web app where AI agents read, write, run and remember inside your own projects. One process, one SQLite file, your machine. The only thing that leaves it is the model traffic you choose to send.

WHAT IT IS

A SolidJS web app in front of an Effect-TS server. You open a project directory and talk to an agent in a chat: it **explores your codebase**, writes and edits files, runs your tests and linters, searches the web when you ask it to, uses any MCP server you connect, and keeps a memory of your project between sessions.

WHAT IS NOT IT

Not a cloud IDE, not a model provider, not a place your code is hosted. There is no account to create and no subscription. You bring the machine; the app brings the agent, the tools, the diffs and the history.

How it works

1. **Point it at a project.** The server reads that directory and nothing else.
2. **Write a sentence.** “Fix the flaky session test”, “add a migration for the new column”, “summarise my open pull requests”. No commands, no syntax.
3. **The agent works through tools** — read, grep, glob, edit, write, bash — each one behind a permission rule you set, and each one visible in the transcript as it happens.
4. **Changes arrive as a diff.** Nothing is applied behind your back: you read the added and removed lines and accept or discard them.
5. **Or let it run on its own.** A background task is a session with a schedule: it re-runs on a cron expression, with its own agent, its own model, and its own log.

THE MODEL IS YOURS

DiveeComposer is a router, not a model: it resolves every turn to the best free model that is actually reachable right now, and remembers which models answer. A model that fails is left alone for a day. Bring your own key for anything else — Anthropic, OpenAI, Google, OpenRouter, Ollama on your own box.

THE SHAPE OF IT

`bun run dev:both` starts the server and the app. The browser is just a client: the server owns the sessions, the tools, the permissions and the database. Behind a proxy, on your LAN, or over SSH — the same build.

The limits, stated plainly.

A self-hosted agent is a different bargain from a hosted one. Here is what you are actually getting, and where it stops.

THE MODEL

- **Free models are free models.** The bundled free tier is genuinely free and genuinely good enough to work with — and genuinely below a frontier model on hard reasoning. The app does not pretend otherwise.
- **The free tier moves.** Models are added and retired. The router re-resolves every turn, and a model that errors or answers with nothing is not used again for a day.
- **Reachability is not a given.** "Free" in the catalogue may still need a key. The router prefers what is actually reachable — which is why the bundled tier is the one it picks.
- **Your key, your bill.** Point it at your own provider and the cost, the rate limits and the data path are yours.

THE AGENT

- **It can be confidently wrong.** It writes code the way a good junior engineer writes code: fast, plausible, occasionally wrong. The diff is the safety net — read it.
- **It only knows what you give it.** It searches as it goes rather than holding an index of your whole repository. Ask it to look, and it looks.
- **Permissions are the real control.** Per tool, per pattern: "allow `git *`", "deny `rm *`", "ask on `external_directory`". A deny is a deny.
- **Background tasks are still agents.** A scheduled task runs unattended with the permissions it has. Give one the power to push, and it will push unattended.

THE PRODUCT, TODAY

- **Early.** The v2 redesign is mid-migration: some surfaces are the new components, some are still the old ones. The app is usable and improving, not finished.
- **Web only.** The terminal UI was removed on purpose. Everything happens in the browser.
- **One password.** Access is a single shared password — enough for a machine on your network, not a multi-user identity system. Put it behind your own proxy for anything more.
- **No built-in TLS.** Terminate it at nginx or Caddy. Sessions live in your SQLite file, so protect the file, not just the URL.

OPERATING IT

- **One SQLite file is the source of truth.** Sessions, messages, tasks and memory live in it. Back it up like you back up your code.
- **A restart is safe.** Tasks that were running when the process died are picked back up: stale ones are marked interrupted, and a task's call journal is replayed instead of repeated.
- **Remote sessions run remotely.** Over SSH the agent works on the remote machine; `ctx_local` runs a single tool call back on your laptop.
- **Nothing is required from us.** No telemetry, no account, no sync. If you turn on crash reporting, it goes where you pointed it.